Day 34 – Queue Problems

🔥 1. Reverse a Queue

💡 Idea:

Queue FIFO hota hai → reverse karne ke liye Stack use karte hain (LIFO)

🧠 Steps:

Queue → Stack me daalo

Stack → wapas Queue me daalo

✅ Code:

```
import java.util.*;

public class ReverseQueue {
    public static void reverseQueue(Queue<Integer> q) {
        Stack<Integer> st = new Stack<>();

        // Step 1: Queue -> Stack
        while (!q.isEmpty()) {
            st.push(q.poll());
        }

        // Step 2: Stack -> Queue
        while (!st.isEmpty()) {
            q.add(st.pop());
        }
    }

    public static void main(String[] args) {
        Queue<Integer> q = new LinkedList<>();
        q.add(10);
        q.add(20);
        q.add(30);

        reverseQueue(q);

        System.out.println(q); // [30, 20, 10]
    }
}
```

🔥 2. Implement Queue using Stack

💡 Idea:

2 stack use karte hain:

stack1 → input

stack2 → output

✅ Code:

```
1. import java.util.*;

class QueueUsingStack {
    Stack<Integer> s1 = new Stack<>();
    Stack<Integer> s2 = new Stack<>();

    void enqueue(int x) {
        s1.push(x);
    }

    int dequeue() {
        if (s2.isEmpty()) {
            while (!s1.isEmpty()) {
                s2.push(s1.pop());
            }
        }
        return s2.pop();
    }

    public static void main(String[] args) {
        QueueUsingStack q = new QueueUsingStack();
        q.enqueue(10);
        q.enqueue(20);
        q.enqueue(30);

        System.out.println(q.dequeue()); // 10
    }
}
```

🔥 3. First Non-Repeating Character in Stream

💡 Idea:

Queue → store characters

Frequency array → count track

✅ Code:

```
1. import java.util.*;

public class FirstNonRepeating {
    public static void main(String[] args) {
        String str = "aabc";
        Queue<Character> q = new LinkedList<>();
        int freq[] = new int[26];

        for (char ch : str.toCharArray()) {
            q.add(ch);
            freq[ch - 'a']++;

            while (!q.isEmpty() && freq[q.peek() - 'a'] > 1) {
                q.poll();
            }

            if (q.isEmpty())
                System.out.print("-1 ");
            else                 System.out.print(q.peek() + " ");
        }
    }
}
2.
```

Output:

a -1 b b

🔥 4. Sliding Window Maximum

💡 Idea:

Deque use karte hain

Har window ka max efficiently nikalte hain

✅ Code:

```
import java.util.*;

public class SlidingWindowMax {
    public static void maxWindow(int arr[], int k) {
        Deque<Integer> dq = new LinkedList<>();

        for (int i = 0; i < arr.length; i++) {

            // Remove out of window
            if (!dq.isEmpty() && dq.peek() == i - k) {
                dq.poll();
            }

            // Remove smaller elements
            while (!dq.isEmpty() && arr[dq.peekLast()] < arr[i]) {
                dq.pollLast();
            }

            dq.offer(i);

            // Print result
            if (i >= k - 1) {
                System.out.print(arr[dq.peek()] + " ");
            }
        }
    }

    public static void main(String[] args) {
        int arr[] = {1, 3, -1, -3, 5, 3, 6, 7};
        int k = 3;

        maxWindow(arr, k);
    }
}
```

Output:

3 3 5 5 6 7

🔥 5. Circular Tour (Petrol Pump Problem)

💡 Idea:

Total petrol ≥ total distance → possible

Otherwise not possible

✅ Code:

```
1. public class PetrolPump {
    static class Pump {
        int petrol, distance;

        Pump(int p, int d) {
            petrol = p;
            distance = d;
        }
    }

    public static int tour(Pump arr[], int n) {
        int start = 0, balance = 0, deficit = 0;

        for (int i = 0; i < n; i++) {
            balance += arr[i].petrol - arr[i].distance;

            if (balance < 0) {
                start = i + 1;
                deficit += balance;
                balance = 0;
            }
        }

        return (balance + deficit >= 0) ? start : -1;
    }
}
```